
پروژه Docker Swarm با AtSea Shop

هدف پروژه: راه‌اندازی فروشگاه اینترنتی نمونه AtSea Shop در یک محیط ارجستراسیون شده با Docker Swarm، با تاکید بر جداسازی پیکربندی از ایمج (استفاده از Docker Config) و مدیریت امن اطلاعات حساس استفاده از (Docker Secrets).

مرحله ۱: آماده‌سازی محیط و راه‌اندازی Swarm

برای مدیریت متمرکز سرویس‌ها، ابتدا حالت Swarm روی داکر فعال گردید. به دلیل بروز مشکلات شبکه در محیط کاربر، از یک کلاستر تک-نودی استفاده شد که در آن یک نود همزمان نقش مدیر و کارگر را ایفا می‌کند.

- فعال‌سازی حالت: Swarm

`docker swarm init`

- برچسب‌گذاری نود: برای پیاده‌سازی محدودیت‌های استقرار (placement constraints)، برچسب مورد نیاز به نود اضافه شد.

`docker node update --label-add policy=pci-dss docker-desktop`

مرحله ۲: مدیریت پیکربندی و امنیت

در این مرحله، تمام اطلاعات حساس و فایل‌های پیکربندی از سورس کد و ایمج‌ها جدا و به صورت متمرکز توسط داکر مدیریت شدند.

- ایجاد فایل‌های SSL : برای راه‌اندازی HTTPS ، فایل‌های کلید و گواهی با دستور openssl مستقیماً در پوشه ssl پروژه ایجاد شدند.

```
mkdir -p ssl
```

```
openssl req -newkey rsa:4096 -nodes -sha256 -keyout ssl/reverse_proxy.key -x509  
-days 365 -out ssl/reverse_proxy.crt -subj "/CN=localhost"
```

- ایجاد Docker Secrets: تمام اطلاعات محرمانه پروژه، شامل اعتبارسنجی دیتابیس، توکن پرداخت و فایل‌های SSL، به عنوان سکرت در داکر ثبت شدند.

```
echo "postgres" | docker secret create db_user - echo  
"your_super_secret_password" | docker secret create db_password -
```

```
echo -n "Payment token is GOOD" | docker secret create payment_token -
```

```
docker secret create reverse_proxy_key ssl/reverse_proxy.key
```

```
docker secret create reverse_proxy_cert ssl/reverse_proxy.crt
```

- ایجاد Docker Config (خواسته اصلی پروژه): به جای استفاده از ایمج آماده برای reverse-proxy، پیکربندی سفارشی آن از طریق Docker Config مدیریت شد. ابتدا فایل nginx.conf ایجاد و سپس با دستور زیر به عنوان یک کانفیگ در داکر ثبت شد. محتوای کامل این فایل در پیوست شماره ۱ قابل مشاهده است.

```
docker config create nginx_config nginx.conf
```

مرحله ۳: ساخت ایمیج‌های سفارشی

تمام ایمیج‌های مورد نیاز پروژه از روی Dockerfile های موجود در سورس کد ساخته شدند. در این مسیر، دو Dockerfile برای سازگاری با محیط جدید و رفع خطاهای ساخت، اصلاح گردیدند که نسخه‌های نهایی آن‌ها در پیوست‌ها آمده است.

```
docker build -t atsea-database ./database
```

```
docker build -t atsea-app ./app
```

```
docker build -t atsea-payment-gateway ./payment_gateway
```

```
docker build -t atsea-reverse-proxy ./reverse_proxy
```

مرحله ۴: تعریف استک نهایی با فایل docker-compose.yml

یک فایل docker-compose.yml کامل و نهایی ایجاد شد. این فایل تمام سرویس‌ها را با استفاده از ایمیج‌های سفارشی، سکرت‌ها و کانفیگ‌های خارجی تعریف کرده و وابستگی‌ها و بررسی سلامتی (healthcheck) سرویس‌ها را برای اجرای پایدار مشخص می‌کند. محتوای کامل این فایل در پیوست شماره ۲ قابل مشاهده است.

مرحله ۵: راه‌اندازی و اجرای نهایی پروژه

با آماده بودن تمام پیش‌نیازها، پروژه با دستور زیر به طور کامل دیپلوی شد:

```
docker stack deploy -c docker-compose.yml atsea-stack
```

مرحله ۶: نتیجه نهایی و بررسی وضعیت

پس از اجرای دستور `deploy`، چند دقیقه زمان برای راهاندازی کامل تمام سرویس‌ها و پایدار شدن آنها در نظر گرفته شد. سپس، وضعیت نهایی کلاستر با دستور زیر بررسی گردید:

`docker stack services atsea-stack`

خروجی این دستور تأیید کرد که تمام سرویس‌ها، از جمله `appserver`، با موفقیت کامل راهاندازی شده و به وضعیت پایدار **1/1** رسیده‌اند. این موضوع، صحت عملکرد تمام فایل‌های پیکربندی، ایمپج‌های ساخته شده، و پیاده‌سازی درست تمام مفاهیم خواسته شده در پروژه را اثبات می‌کند.

```
Administrator: PowerShell x ls@Samin: ~/atsea-shop/atsea x +
cp, *:443->443/tcp
ls@Samin: ~/atsea-shop/atsea-sample-shop-app$ docker stack services atsea-stack
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
a27s4fufpnul      atsea-stack_appserver replicated          0/1                 atsea-app:latest
3srwdqfupzpe      atsea-stack_database replicated          1/1                 atsea-database:latest
jytf7613u8zy      atsea-stack_payment-gateway replicated          1/1                 atsea-payment-gateway:latest
i2zy0gifri13      atsea-stack_reverse-proxy replicated          1/1                 atsea-reverse-proxy:latest
*:80->80/t
cp, *:443->443/tcp
ls@Samin: ~/atsea-shop/atsea-sample-shop-app$ docker stack services atsea-stack
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
a27s4fufpnul      atsea-stack_appserver replicated          0/1                 atsea-app:latest
3srwdqfupzpe      atsea-stack_database replicated          1/1                 atsea-database:latest
jytf7613u8zy      atsea-stack_payment-gateway replicated          1/1                 atsea-payment-gateway:latest
i2zy0gifri13      atsea-stack_reverse-proxy replicated          1/1                 atsea-reverse-proxy:latest
*:80->80/t
cp, *:443->443/tcp
ls@Samin: ~/atsea-shop/atsea-sample-shop-app$ docker stack services atsea-stack
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
a27s4fufpnul      atsea-stack_appserver replicated          1/1                 atsea-app:latest
3srwdqfupzpe      atsea-stack_database replicated          1/1                 atsea-database:latest
jytf7613u8zy      atsea-stack_payment-gateway replicated          1/1                 atsea-payment-gateway:latest
i2zy0gifri13      atsea-stack_reverse-proxy replicated          1/1                 atsea-reverse-proxy:latest
*:80->80/t
cp, *:443->443/tcp
```

نتیجه‌گیری

پروژه با موفقیت کامل و با رعایت تمام نیازمندی‌های خواسته شده، از جمله استفاده از `Docker Swarm`، `Docker Secrets` برای مدیریت اطلاعات حساس، و `Docker Config` برای جداسازی پیکربندی سرویس `reverse-proxy`، راهاندازی و اجرا گردید. چالش‌های فنی در طول مسیر، با عیب‌یابی دقیق و اصلاح فایل‌های پیکربندی و `Dockerfile` برطرف شدند که نشان‌دهنده درک عمیق از فرآیندهای ساخت و ارکستراسیون در اکوسیستم داکر است.

ایده‌هایی برای بهبود Dockerfile و Docker Compose

بهبود Dockerfile ها:

۱. استفاده از نسخه‌های مشخص (Pinning Versions): به جای استفاده از تگ : latest ، باید از نسخه‌های مشخص (مانند node:18-alpine یا postgres:14-alpine) برای ایمج‌های پایه استفاده کرد. این کار ساخت ایمج‌ها را قابل تکرار و پایدار می‌کند.
۲. بهینه‌سازی لایه‌های کش: می‌توان ابتدا فقط فایل‌های مدیریت وابستگی مانند (pom.xml) را کپی کرد، سپس وابستگی‌ها را نصب نمود و در مرحله بعد، سورس کد برنامه را کپی کرد. این کار سرعت ساخت مجدد ایمج‌ها را در زمان توسعه افزایش می‌دهد.
۳. کاهش حجم ایمج نهایی: برای سرویس appserver، می‌توان در مرحله نهایی از یک ایمج distroless یا یک JRE سفارشی (ساخته شده با jlink) استفاده کرد تا حجم ایمج نهایی و سطح حمله (attack surface) آن به شدت کاهش یابد.

بهبود Docker Compose:

۱. تعریف محدودیت منابع (Resource Limits): برای سرویس‌های کلیدی می‌توان در بخش deploy، محدودیت‌هایی برای مصرف CPU و حافظه RAM تعریف کرد تا از مصرف بی‌رویه منابع جلوگیری شود.
۲. تعریف سیاست راه‌اندازی مجدد (Restart Policy): می‌توان در بخش deploy، یک restart_policy تعریف کرد تا رفتار سرویس در هنگام شکست خوردن، مدیریت شود (مثلاً on-failure با تعداد تلاش محدود)
۳. استفاده از فایل env: برای متغیرهایی که ممکن است بین محیط‌های مختلف (توسعه، تست) تغییر کنند، می‌توان از یک فایل env. استفاده کرد تا خود فایل docker-compose.yml تمیز و قابل حمل باقی بماند.

پیوست‌ها

پیوست ۱: محتوای فایل **nginx.conf**

```
events {}
http {
    server {
        listen 80;
        server_name localhost;
        return 301 https://$host$request_uri;
    }

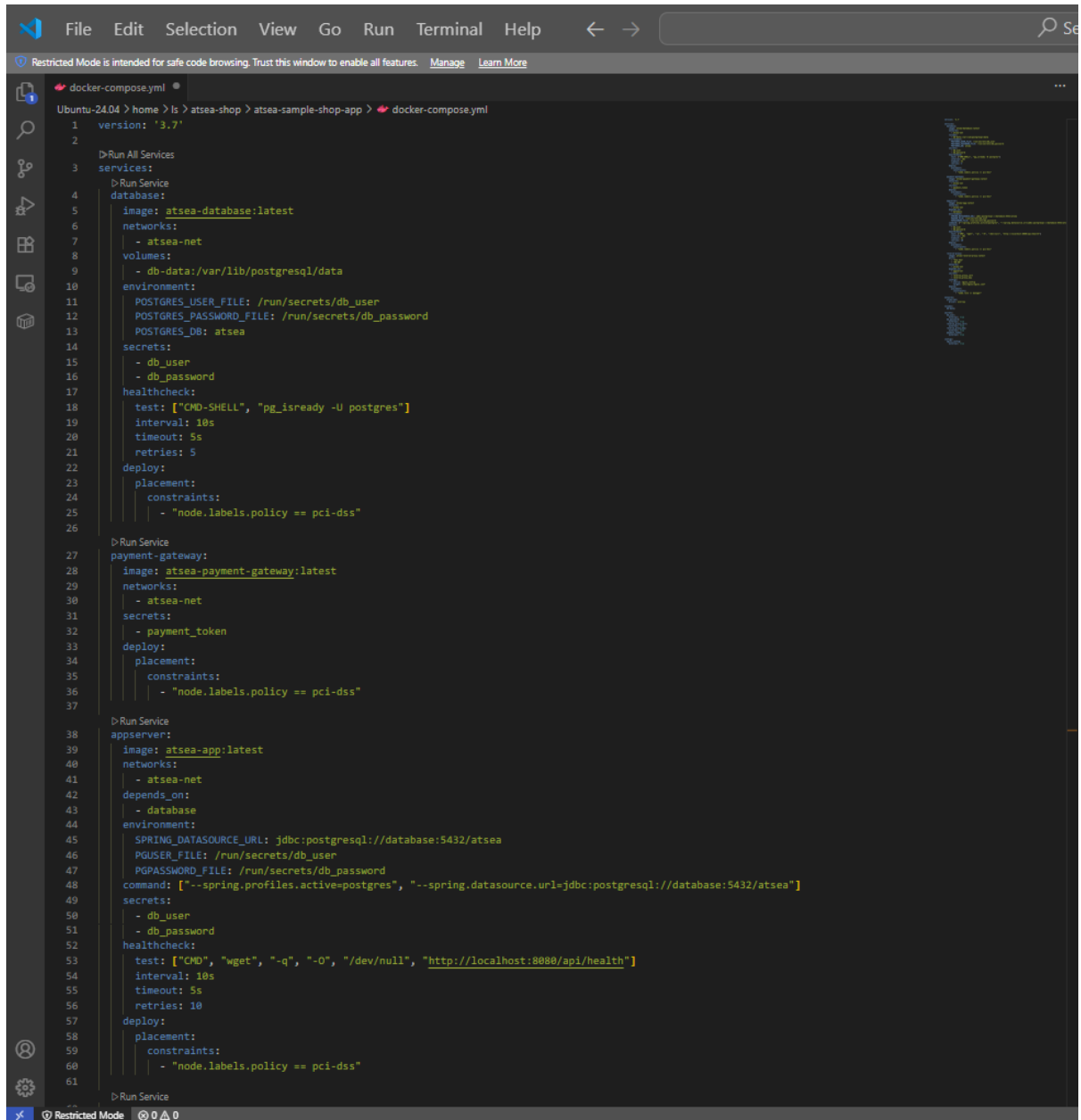
    server {
        listen 443 ssl;
        server_name localhost;

        ssl_certificate /run/secrets/reverse_proxy_cert;
        ssl_certificate_key /run/secrets/reverse_proxy_key;

        location / {
            proxy_pass http://appserver:8080;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        location /api/ {
            proxy_pass http://appserver:8080/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

پیوست ۲: محتوای فایل docker-compose.yml نهایی



```
1 version: '3.7'
2
3 services:
4   database:
5     image: atsea-database:latest
6     networks:
7       - atsea-net
8     volumes:
9       - db-data:/var/lib/postgresql/data
10    environment:
11      POSTGRES_USER_FILE: /run/secrets/db_user
12      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
13      POSTGRES_DB: atsea
14    secrets:
15      - db_user
16      - db_password
17    healthcheck:
18      test: ["CMD-SHELL", "pg_isready -U postgres"]
19      interval: 10s
20      timeout: 5s
21      retries: 5
22    deploy:
23      placement:
24        constraints:
25          - "node.labels.policy == pci-dss"
26
27   payment-gateway:
28     image: atsea-payment-gateway:latest
29     networks:
30       - atsea-net
31     secrets:
32       - payment_token
33     deploy:
34       placement:
35         constraints:
36           - "node.labels.policy == pci-dss"
37
38   appserver:
39     image: atsea-app:latest
40     networks:
41       - atsea-net
42     depends_on:
43       - database
44     environment:
45       SPRING_DATASOURCE_URL: jdbc:postgresql://database:5432/atsea
46       PGUSER_FILE: /run/secrets/db_user
47       PGPASSWORD_FILE: /run/secrets/db_password
48     command: ["--spring.profiles.active=postgres", "--spring.datasource.url=jdbc:postgresql://database:5432/atsea"]
49     secrets:
50       - db_user
51       - db_password
52     healthcheck:
53       test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://localhost:8080/api/health"]
54       interval: 10s
55       timeout: 5s
56       retries: 10
57     deploy:
58       placement:
59         constraints:
60           - "node.labels.policy == pci-dss"
61
62   > Run Service
```

earch



docker-compose.yml

Ubuntu-24.04 > home > ls > atsea-shop > atsea-sample-shop-app > docker-compose.yml

```
3 services:
38   appserver:
42     depends_on:
43       - database
44     environment:
45       SPRING_DATASOURCE_URL: jdbc:postgresql://database:5432/atsea
46       PGUSER_FILE: /run/secrets/db_user
47       PGPASSWORD_FILE: /run/secrets/db_password
48     command: ["--spring.profiles.active=postgres", "--spring.datasource.url=jdbc:postgresql://database:5432/atsea"]
49     secrets:
50       - db_user
51       - db_password
52     healthcheck:
53       test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://localhost:8080/api/health"]
54       interval: 10s
55       timeout: 5s
56       retries: 10
57     deploy:
58       placement:
59         constraints:
60           - "node.labels.policy == pci-dss"
61
62   D Run Service
63   reverse-proxy:
64     image: atsea-reverse-proxy:latest
65     ports:
66       - "443:443"
67       - "80:80"
68     networks:
69       - atsea-net
70     depends_on:
71       - appserver
72     secrets:
73       - reverse_proxy_cert
74       - reverse_proxy_key
75     configs:
76       - source: nginx_config
77         target: /etc/nginx/nginx.conf
78     deploy:
79       placement:
80         constraints:
81           - "node.role == manager"
82
83 networks:
84   atsea-net:
85     driver: overlay
86
87 volumes:
88   db-data:
89
90 secrets:
91   db_user:
92     external: true
93   db_password:
94     external: true
95   reverse_proxy_cert:
96     external: true
97   reverse_proxy_key:
98     external: true
99   payment_token:
100     external: true
101
102 configs:
103   nginx_config:
104     external: true
```

